

控制层和方向层的分野：OpenProse、Natural-Language Agent Harnesses 与 Attractor-Guided Engineering 的比对分析

原创 可逆计算 可逆计算 2026年6月4日 07:14 北京

相关项目：OpenProse ([@openprose/reactor](#))、NLAHs (Natural-Language Agent Harnesses <https://arxiv.org/abs/2603.25723>)、AGE 模板 <https://github.com/entropy-cloud/attractor-guided-engineering-template>、nop-chaos-flux (AGE 的生产级实施实例) AGE 方法论系列文章：[Attractor Before Harness: AI 大规模开发的方法论](#)、[从 Spec-Driven Development 到 AGE](#)

起点

OpenProse、Natural-Language Agent Harnesses (NLAHs) 和 Attractor-Guided Engineering (AGE) 都在试图解决同一个问题：AI agent 的产出不可靠，怎么办？

前两者在控制层给出答案——OpenProse 用确定性 runtime 让目标可被精确维护，NLAHs 用自然语言文档让 harness 策略可被审查。AGE 追问的则是控制层之前的问题：控制到底应该维护什么结构？

如果方向层不存在，更好的 runtime 和更清晰的 harness 只会让结构整体漂移。

OpenProse 从**运行时抽象**出发——设计一种让 AI agent 长期维护"持续为真"目标的合约语言和运行引擎。

NLAHs 从**表示媒介**出发——将 agent harness 策略从 tangled controller code 外化为可编辑的自然语言文档，由共享运行时 IHR 解释执行。

AGE 从**工程过程**出发——设计一种让人类和 AI 协作时系统持续向稳定结构收敛的文档-流程体系。

OpenProse 问的是：怎样让一个目标不变。NLAHs 问的是：怎样让一次 agent run 按可审查的策略执行。AGE 问的是：系统被推偏之后，为什么还会回到正确方向。

约定：本文中 "NLAHs" 指论文提出的整体框架 (Natural-Language Agent Harnesses)，"NLAH" 指单份策略文档，"IHR" 指共享运行时 (Intelligent Harness Runtime)。

Fixed Point $\neq$ Attractor

OpenProse 和 AGE 的核心抽象对应两个不同的数学对象。

Fixed Point (不动点)： $f(x) = x$ 。系统放在这个点上就不动了。判定条件是严格的等式。不动点可以是稳定的（附近的点向它收敛），也可以是不稳定的（附近的点远离它）。Fixed point 不蕴含 basin of attraction 的概念。

Attractor (吸引子)：一个集合（点、极限环、奇怪吸引子），使得周围一个区域（吸引域，basin of attraction）内的所有初始状态都会随时间演化向它收敛。关键性质不是"放上去不动"，而是"从附近出发会被拉回来"。吸引子不必是一个点，可以是一个流形。系统不需要精确到达它——只要在 basin 内，轨迹在收敛就行。

OpenProse 的指纹比对 `fingerprint == fingerprint` 是 fixed point 检测：输出和之前一样吗？一样就不动了。

AGE 的"系统向稳定结构收敛"可按 attractor 判据理解：系统不需要精确到达某个理想状态，只要控制机制持续把它拉回稳定结构附近。

一个是状态判定——"到了没有"。一个是过程判定——"方向对不对"。

Control Target $\neq$ Attractor: 本体论区别

收敛机制（fixed point vs attractor）的区分之外，还有一个更深的本体论差异：**控制目标 vs 吸引子**。

控制目标 (Control Target)

控制框架已经建立后，定义"控制应该朝向哪里"。它假设：

- 控制框架本身已经存在
- 执行者会按照指令行动
- 偏离时通过**重新执行指令**来纠正

控制目标是外部强加的 specification，**驱动**执行者去符合它。

数学吸引子 (Attractor)

吸引子由系统自身动力学产生——周围区域内的所有初始状态都随时间演化向它收敛。关键性质：

- 它是**涌现的**——由系统动力学定义，不是外部强加的指令
- **扰动-恢复**——推走它，动力学会把它拉回来
- 系统**不需要**"知道"它——水不需要知道洛伦兹吸引子就按它运动
- 它是**方程定义的流形**——不枚举每个正确状态，只定义约束关系

吸引子为控制提供**最终因**（final cause）。它在控制之前，不在控制之内。

注意：AGE 的吸引子只是一种类比，它不是自然涌现的数学对象，而是工程化构造的语义场：人定义结构约束，流程把这些约束转化为跨会话的恢复动力学。它的判据不是"没有外部设计"，而是"扰动后的恢复是否由持久结构场驱动，而非由一次性指令驱动"。

打个比方

操作手册告诉工人怎么组装零件。工人按手册做，偏离了对照手册纠正。手册是控制目标——它需要工人"知道"它并主动执行。

重力井是吸引子——不管你怎么推物体，物体的动力学自然把它拉回来。物体不需要"知道"重力井的存在。

OpenProse 和 NLAHs 的不变量都是控制目标

OpenProse 的 Responsibility（Goal/Maintains/Continuity）是外部定义的 specification。运行时通过指纹比对检测偏离，偏离了就重新 render。这是控制目标 + 确定性执行。收敛靠 runtime 强制重新执行 specification。

NLAHs 的 harness pattern（contract + stages + state / validation / recovery / stopping rules）告诉 IHR 应该怎么组织一次 task run。IHR 不是硬编码控制器，而是用 runtime policy 解释 NLAH，把条款落实为 child-agent calls、state updates、validation gates 和 artifact contracts。如果 agent 偏离，起作用的是 validation / recovery / stopping rules 等控制目标，不是系统动力学自然收敛。"behavior is flexible but still policy-guided" 不是吸引子语义，是**运行时语义约束下的弹性执行**。

AGE 的 owner-doc 体系在类比意义上具备吸引子性质

1. **扰动-恢复动态**：AI 改了代码（扰动）→ test/audit 发现偏离 → 修正 → 系统回到结构附近。跨会话的扰动-检测-纠正循环构成了收敛动力学。
2. **隐式定义的流形**：[docs/architecture/](#) 不是枚举每个正确状态，而是定义约束方程。满足约束的状态构成流形，局部实现多样，整体被拉向同一类结构。
3. **跨会话涌现**：上百次 commit、几十个 plan、多次审计叠加后的长期收敛——这是"系统行为"，不是"某次执行"。

AGE 的恢复同样依赖文档作为真值源。区别不只是粒度和时间尺度，而是**约束对象不同**：控制层指令约束一次执行如何完成；方向层约束未来所有执行结果允许落入的结构区域。

机制层的结构性平行

三个体系的机制层存在结构性平行：

	OpenProse	NLAHs	AGE
真值源	Responsibility (Goal/Maintains)	NLAH (contract + run rules)	Owner docs (docs/architecture/)
解释者	OpenProse reactor	IHR runtime	AI agent 读文档
执行单元	render	task run	plan（重大变更时）
检测	指纹比对	validation gates	CI + audit
关闭条件	postcondition + receipt	completion gates + artifacts	Closure Gates + 独立 closure audit
纠正	重新 render	retry / recovery / honest stop	修正代码，重读文档

机制槽位相似，但语义不同。这个表只说明三者都是控制系统，不说明它们控制的是同一类对象。OpenProse 和 NLAHs 让一次执行更可控；AGE 判断这些执行累积后是否仍指向同一个长期结构。

差异不在机制层，而在机制层之上的**方向层是否存在**。OpenProse 的 Responsibility 和 NLAHs 的 NLAH 都是**自包含的 specification**——它们定义了自己的完成条件（指纹相等 / contract 满足），不需要外部引用"系统应该向什么结构收敛"。AGE 也有完成条件（CI gates、audit 通过），但 owner docs 的不可替代作用是：当局部完成条件与长期结构方向冲突时，提供裁决依据。CI 可以判定某次 import 是否通过；owner-doc 决定为什么 `flux-react` 不能反向依赖 `renderer` 包。前者是控制门，后者是方向层。

这就是方向层 vs 控制层的区别：不是机制不同，而是**控制之上有没有一个独立的、持久的目标结构为所有控制提供最终因**。

三者的本体论位置

三者的关系不是层次关系（project-level attractor $\supset$ run-level attractor），而是**本体论差异**：



AGE 文章的核心论点正是：**方向层先于控制层**。没有定义"朝哪里收敛"，控制机制就没有统一意义。NLAHs 和 OpenProse 都在控制层做创新——NLAHs 用 NL 提高策略的可审计性和可移植性，OpenProse 用确定性 runtime 提高执行的精确性和成本效率。但它们都不能回

答方向层的问题：**系统应该向什么结构长期收敛？** 一个可证伪的表述是：如果移除 AGE 的 owner-doc 优先级链、日志和 bug 蒸馏后，系统仍能在长期重构中保持同等架构方向，那么 AGE 的方向层主张就是多余的。

三个体系的前提假设

OpenProse：会话终将结束，责任不应随之消失

OpenProse 的核心隐喻来自 React。它把 AI agent 的每一次执行建模为一个 render。

三个关键假设：

1. AI 的价值在于 bounded render——有限、可审计的单次推理
2. 持续性的来源不是会话，而是 world-model 的持久状态和 receipt 链
3. 成本与"意外"成正比，与时间无关——如果没有变化，render 不执行

这是 runtime-first 的世界观：先定义好"什么在运行时保持为真"，再考虑工程过程。

NLAHs：策略不应埋在代码里

NLAHs (Natural-Language Agent Harnesses) 的核心隐喻来自可编辑的策略文档。

NLAH+IHR 体系把 agent harness 拆成四层栈。

三个关键假设：

1. Harness 策略可以从 tangled controller code 中分离出来，用可编辑的自然语言表达
2. NL 携带策略，code 携带机制——精确操作留在代码里，可检查的策略留在 NL 里
3. 共享 runtime (IHR) 可以为不同的 NLAH 提供统一的执行基底

这是 representation-first 的世界观：策略的表示媒介决定了它的可审查性、可移植性和可分析性。

AGE：先定义系统要回到哪里，再讨论怎么回去

AGE 的核心隐喻来自动力系统理论。

三个关键假设：

1. AI 是结构上不同于人类的扰动源——高频、高振幅、无持续方向感
2. Attractor 必须先于 harness 存在——"朝哪里纠正"先于"如何纠正"
3. 文档是 attractor 的载体，代码是 attractor 的瞬时投影——两者都不是 attractor 本身

这是 process-first 的世界观：先定义好"系统应该向什么结构收敛"，再设计运行时的行为。

分歧一览

维度	OpenProse	NLAHs	AGE
核心问题	如何让 AI 持续维护一个为真的目标？	如何将 harness 策略外化为可审查的 NL？	如何让系统在 AI 高频扰动下持续向稳定结构收敛？
问题域	单个 responsibility 的生命周期	单次 task run 的 harness 策略	整个仓库的演化轨迹
时间尺度	单次 render（分钟）到持续 serve（天）	单次 task run（分钟到小时）	单次 commit（分钟）到项目演化（月/年）
不变量性质	控制目标（specification + fixed point）	控制目标（specification + 软约束）	工程化方向场 (engineered attractor-like structure)
稳定性概念	Fixed point ——指纹相等即不动	Policy conformance ——符合策略即正确	Attractor ——在 basin 内即收敛
"对了"的含义	输出与期望精确匹配（二值）	执行路径符合 NLAH 规定的策略	系统在收敛方向上（方向性）
控制机制	确定性指纹比对（零 LLM）	IHR 解释 NL policy（依赖 LLM）	扰动-恢复动力学（test→audit→fix）
策略执行	Runtime 强制	Runtime + NL 指导	制度化流程诱导，而非 runtime 强制

核心抽象对比

最小原语

OpenProse 的 Responsibility 是一个自包含的计算单元，有明确的输入边界（Requires）、输出边界（Maintains）、目标（Goal）和连续性约束（Continuity）。执行产生一个 receipt——内容寻址、链式可验证的决策证明。

NLAHs 的 NLAH 文档 是一个可编辑的策略描述，定义了 task run 的 lifecycle policy：contract、stages、roles、state rules、verification rules、recovery rules 和 stopping conditions。NLAH 不是"告诉模型怎么回答"，而是"告诉 runtime 怎么组织一次多步执行"。

IHR 的执行语义不是把 NLAH 编译成代码，也不是让一个 agent 读完文档后自由发挥。它用固定 runtime-policy prompt 把 base agent 变成 parent orchestrator：parent 读取 NLAH，把 contract、stages、state rules、validation rules、recovery rules 和 stopping conditions 落实为 child-agent task packets、handoff、状态文件、artifact gates 和 completion checks。阶段内部仍由 child agent 智能执行；测试、parser、validator、benchmark adapter 这类精确操作由 scripts/adapters 承担。

AGE 的 Attractor 不是一个对象，是一个结构性质，由三层组成：

- **结构层**：少量高层不变量（如包依赖方向 `flux-core → ... → flux-renderers`、七种原语闭集、编译优先管道）
- **载体层**：承载这些不变量的可审计文档（`docs/architecture/`）
- **实现层**：当前代码中体现这些不变量的部分

Responsibility 和 NLAH 都是**实例**——你可以数出"这个项目有 23 个 responsibility"或"这个 benchmark 有 1 个 NLAH"。Attractor 是一个**性质**——你不能数出"这个项目有 5 个 attractor"，它更像"这个系统的吸引子是一个低维流形"。

三个体系都有把 AI 行动收束成可关闭单元的机制：OpenProse 的 render、NLAHs 的 task run、AGE 的 plan。Render 的闭包由输入指纹、postcondition 和 receipt 决定；task run 的闭包由 NLAH 的 task contract、validation rules、artifact contracts 和 stopping conditions 决定；plan 的闭包由 `Plan Status`、`Current Baseline`、`Goals`、`Non-Goals`、execution/proof checklist、`Closure Gates` 和独立 closure audit 决定。

Plan 不是待办列表，而是一次重大变更能否关闭的证据结构：先核对当前基线，限定目标与非目标，执行中逐项证明，最后由独立审计确认 scope 内事项已经完成或被诚实移出。它和 owner-doc 的关系也不是每次修改文档，而是用 owner-doc 判断当前变更的边界和关闭条件；只有当变更改变 live baseline、public contract 或 owner behavior 时，closure 才要求同步文档。

NLAH 和 AGE plan 在机制形态上接近：都用自然语言定义 contract、阶段、证据和停止条件，都反对只靠 agent 自述完成。但两者关闭的对象不同：NLAH 关闭的是一次 run，证明执行轨迹满足 harness policy；plan 关闭的是一次变更，证明当前 scope 的结果、证据和文本状态都足以被独立审计接受。

	NLAH	AGE plan
对象	一次 agent run 的 harness policy	一次重大变更的 closure contract
执行结构	IHR parent orchestrator + child agents	实现 agent / 人类按 plan 执行，独立 reviewer 或 fresh subagent 关闭审计
证据载体	STATE_ROOT、artifacts、run logs	plan checklist、proof、verification output、daily log
停止条件	task contract、artifact contract、stopping conditions	Closure Gates、scope 内 checklist、deferred adjudication、closure audit
失败后处理	run 内 retry / recovery / honest stop	不关闭、补 proof、改状态、拆出 non-blocking follow-up 或新 plan

状态模型

	OpenProse	NLAHs	AGE
状态表示	内容寻址的 world-model（形式化）	文件系统状态（STATE_ROOT + artifacts）	代码 + 文档 + 日志（非形式化）
判定机制	确定性指纹函数（canonicalizer）	validation gates + artifact contracts	CI 命令 + 人工审查
历史记录	Receipt ledger（链式可验证）	Artifact 文件 + 日志（单次 run 内）	日志 + Bug Notes + Git history
稳定性语义	Fixed point 检测（hash == hash）	Policy conformance（符合策略）	Attractor 收敛（在 basin 内且方向正确）
跨会话记忆	Receipt chain（链式可验证）	无（单次 run 内的文件状态）	有（logs、bugs、discussions、plans）

技术实现对比

OpenProse 是严格的两阶段架构：编译期用 LLM 产生确定性工件（Forme DAG、canonicalizer、postcondition validators），运行期从不调用 LLM，所有决策都是确定性的。AGE 没有"编译一次然后运行"的分离，验证持续嵌入在每次变更中——typecheck → build → lint → test → 审计 → 日志，永远在验证方向。NLAHs 处于两者之间：NL policy 与确定性 hooks 分离，但 IHR 运行时依赖 LLM 解释策略。

AGE 明确识别了**自验证陷阱**：AI 从同一上下文生成代码和所有评判材料（类型、测试、文档、完成总结），如果理解有偏差，所有"验证证据"会朝同一个方向偏离。对策是强制生成和评判分离——闭包审计必须由新上下文执行。OpenProse 通过编译期冻结确定性函数来绕过同一个问题。NLAHs 在论文的评估场景中靠 benchmark autograder 规避，但在生产部署中同样需要外部验证机制来避免自验证陷阱。

对"一致性"的理解——三种语义

这是三个体系最深层的分歧点。

OpenProse 的 fixed-point 一致性：每个 responsibility 的 render 产出 world-model，通过 canonicalizer 得到指纹。指纹和上次一样，系统"一致"。判定是二值的：相等或不相等，没有"差不多一致"。组合层面，每个 responsibility 都在自己的不动点上，DAG 拓扑正确，则整个系统一致。自底向上，每个局部不动点组合成全局不动点。

NLAHs 的 policy-conformance 一致性：agent 的执行路径是否遵循 NLAH 规定的策略。Paper 定义了 pattern-preservation、stage-coverage、artifact-contract compliance 等指标来度量。不是二值的——NLAH 允许灵活执行，但要求策略结构（阶段、角色、验证门、状态规则）被保留。Paper 的 RQ2 发现 NLAH 保留了可识别的工作流结构（workflow preservation 0.63-0.67, stage coverage 0.57-0.82），但信息传递是主要弱点（handoff recall 低至 0.32）。

AGE 的 attractor 一致性：不是二值的。一个系统可以"大部分在 attractor 附近，但某个子系统偏离了"——这不是"不一致"，而是"在 basin 内但偏离中心"。判定标准是方向性的：系统是在向 attractor 收敛，还是在远离？一次 commit 可以通过所有测试（在 basin 内），但如果引入了架构偏移（比如逆转包依赖方向），系统的轨迹就不再向 attractor 收敛了。自顶向下——即使每个模块的测试都通过（每个局部都在不动点），整体仍然可以偏离 attractor。

AGE 的原始论断：_"所有 state-level checks 都能通过，但系统整体在漂移"。nop-chaos-flux 的 Plan 76 是一个实例：移除 `array-editor` 局部状态镜像的尝试产生了 11 个测试失败——不是因为单个 bug，而是因为测试本身已经漂移到了与旧实现的时序耦合状态。每个累积的变更都通过了 review 和 CI，但从 attractor 视角看，测试套件已经漂移了能支持结构性演化的位置。这恰恰是 attractor 概念存在的原因。

这个区分不是术语偏好：

1. Fixed point 方法只能检测"是否精确到达"——对"在收敛方向上但还没到达"和"已经偏离"的区分能力很弱
2. Attractor 方法天然关注轨迹方向——不需要系统精确到达，只需要系统在收敛，而收敛本身就是可持续的
3. Fixed point 在离散状态空间（如 world-model 的指纹）中工作良好；Attractor 在连续状态空间（如架构的演化）中更有表达力
4. 这直接解释了为什么 OpenProse 用 `hash == hash`（离散等式），AGE 用"CI 通过 + 文档一致 + 架构边界完整"（多维收敛信号）

三个维度

对 AI 的定位。 OpenProse 把 AI 建模为有界的 render 函数，角色是**执行器**。NLAHs 把 AI 建模为被 harness policy 组织的执行系统：parent orchestrator 按 runtime policy 解释 NLAH，child agents 执行具体任务，角色分化为**编排者 + 执行者**。AGE 把 AI 建模为高频、高振幅、无持续方向感的扰动，角色是**扰动源 + 执行器**——同时被约束和被使用。

对文档的理解。 OpenProse：*.prose.md 就是程序，如果任何东西与 Markdown 不一致，Markdown 是对的。NLAHs：NLAH 文档是**策略**——规定"怎么做"但不直接"执行"，最可审查的部分从实现细节中分离出来。AGE：文档是 attractor 的**载体层**，代码是 attractor 的**瞬时投影**，载体层定义了它应该回到哪里。

失败处理。 OpenProse：失败产生 failed receipt，即时的、个体的，每个 render 独立记录。NLAHs：NLAH 定义 Recovery Rules（重试、回退、证据），失败的 artifact 保留供分析，但不跨 run 蒸馏。AGE：失败产生结构化 Bug Note，反复出现的失败模式被蒸馏为 guardrails，按渐进自动化阶梯升级（笔记 → 检查清单 → 启发式脚本 → lint 规则 → CI 门控）。

渐进式自动化的实证：nop-chaos-flux 的工具链

AGE 方法论有一个核心论断：_能落实为确定性检查的，会逐步从轨迹数据中抽取为确定性脚本_。

nop-chaos-flux 的工具链为这条论断提供了完整的实证。实际发生的 5 级渐进自动化阶梯：

```
Tier 1: 轨迹捕获 (Bug Notes + Logs)
62+ bug fix notes, 72 daily dev logs

Tier 2: 模式蒸馏 (Architecture Guardrails + Audit Rules)
8 guardrails distilled from bugs, 19 dedicated audit rules

Tier 3: 编码化的审查模式 (Skills/Prompts)
24 reusable audit/review prompt templates

Tier 4: 启发性嫌疑扫描器 (Heuristic Scanners, exit 0)
12 focused audit scanners, informational output

Tier 5: 硬门控自动化 (Hard Gates, CI-blocking, exit 1 on failure)
14 hard gate scripts, 30+ ESLint rules, dependency-cruiser, Semgrep, Husky pre-commit
```

实例 A：构建产物污染源码目录

Tier 1 — Bug #23: `packages/*/src/` 中残留的 `.js` 文件在 Vitest 运行时暗中覆盖了 `.ts` 源码，导致测试失败但根因不在测试代码。共发现 102 个过期文件。

Tier 2 — Guardrail #6 "No Build Artifacts In Source Directories" 被蒸馏进 `docs/references/architecture-guardrails-from-bugs.md`，附有 bug 证据链接。

Tier 5 — `scripts/verify-no-src-artifacts.mjs` 成为硬门控 (CI-blocking)，同时 `scripts/clean-src-artifacts.mjs` 提供清理能力。 `pnpm lint` 链在 ESLint 之前先跑 `clean + verify`。

从 bug 到 CI 门控，完整走完了 5 级。

一个思想实验：如果用 OpenProse 或 NLAHs 管理 nop-chaos-flux

如果把 nop-chaos-flux 的架构不变量建模为 OpenProse responsibilities，会得到运行时精确的变化检测，但无法回答为什么这些不变量应该这样组织、如何演化。如果把 plan 执行建模为 NLAHs，会得到结构化的执行流程，但无法回答 100 次 plan 之后系统是否仍在收敛。两者都是控制层工具；都不覆盖方向层。

AGE 的 owner-doc 系统处理这些问题。OpenProse 的 responsibility 合约不覆盖方向层。NLAHs 的策略文档不覆盖跨会话的轨迹演化。两者都是优秀的控制层工具，但吸引子（方向

层) 不在它们的问题域内。

结论

OpenProse 回答的是："给定一个需要持续为真的目标，如何用 AI 高效、安全、可审计地维护它？"——微观运行时问题。

NLAHs 回答的是："给定一个 tangled 的 controller code，如何把可复用的 harness 策略外化为可审查、可移植、可消融分析的 NL 文档？"——表示媒介问题。

AGE 回答的是："给定一个在 AI 扰动下持续演化的系统，如何保证它不会偏离稳定结构？"——宏观过程问题。

三者守护的是不同形式化/工程化稳定性语义，甚至不同本体论层次。OpenProse 信任算法（确定性函数、内容寻址、链式验证）来维持不动点；NLAHs 信任可审查的 NL 策略（IHR 解释执行、模块可消融）来维持策略遵循；AGE 信任的不是泛泛流程，而是可审计、可累积、可对抗复核、能把失败沉淀为未来约束的过程。OpenProse 和 NLAHs 在控制层做创新；AGE 在方向层做贡献。

真正的分歧在于失败会在哪里累积：OpenProse 把失败局部化到 responsibility，NLAHs 把失败局部化到 task run，AGE 把失败看作跨会话轨迹的信号。

在 nop-chaos-flux 的实践中，AGE 的渐进自动化阶梯已经跑通：62+ bug notes → 8 guardrails → 14 硬门控 + 12 启发性扫描器 + 30+ ESLint 规则。Bug #23（构建产物污染）完整走完了从 bug note 到 CI-blocking 硬门控的 5 级蒸馏。72 个日志文件记录了系统的轨迹，46 个执行计划是局部收敛机制，58 个架构文档的优先级链是 attractor 的载体层。

OpenProse 可以把 AGE 已经明确的局部不变量变成精确的变化检测。NLAHs 可以把 harness 变成可消融对象。两者都是优秀的控制层工具；但吸引子（方向层）不在它们的问题域内。它们可以成为控制层执行器，不能替代方向层裁判。

可以这么说：如果一个项目连续通过 CI、review 和 harness contract，却在 50 次 AI-assisted change 后出现包边界逆转、测试语义耦合和 owner-doc 冲突，那么 OpenProse/NLAHs 的控制层成功了，而 AGE 要处理的问题才刚刚显现。这就是三者的分界线：不是谁更全面，而是谁能看见长期漂移。

控制层能告诉 AI 怎么纠错；方向层决定了什么才叫错。

喜欢作者

可逆计算 · 目录

上一篇

下一篇

